Case Study: RAG-Diary

> An intelligent diary that stores, retrieves and creates memories by meaning. A record of business analysis and system analysis work, from the problem to delivery.

> Thai version: CASE-STUDY.md

1. Document information

Item	Detail
Title	RAG-Diary: analysis and design of an intelligent diary system
Author	Business Analyst / System Analyst (project owner)
Project type	Self-initiated personal project
Prepared	15 October 2026
Roles covered	Business analysis, system analysis, documentation

2. Executive summary

Most people keep memories in several places, in note apps and on paper. Years later they cannot find them, and they cannot recall how they felt at the time. The problem is not a shortage of writing. It is retrieving memories that are tied to feelings.

This project designs and builds a diary that searches by meaning rather than by exact wording. A user types what they want to recall, the system returns the related entries with a similarity score, and the user can hold a conversation with their own notes.

The system then moves from retrieving the past to creating memories worth keeping. A user summarises the mood of a period as text, turns that summary into an illustration, and keeps the moment in a form they can hold.

The result is a working system that covers 15 functional requirements, from writing and semantic search to conversation, mood summaries and per-user data isolation at the database level, together with 25 documents that trace every requirement to a test.

3. Context and problem

Users keep notes in several places, and no single place can be searched by meaning. Reviewing how a period felt means scrolling entry by entry, and the memories worth keeping fade under newer entries.

The pain that actually bites: exact-word search misses near meanings; there is no view of mood over time; there is no way to keep an important period in a tangible form; and there is a real worry that private entries leak when they are processed by an external service.

4. Goals

Four goals shaped the work: retrieve memories by meaning, see the shape of one's own feelings, create a keepsake from an important period, and be confident that private data is isolated and protected.

5. Role and working method

I covered both the business analyst and system analyst roles. The method favours agreeing the shape before writing: every document passes a review of purpose, scope, storyline, depth, tools, traceability and the definition of done before drafting starts.

The work uses established frameworks: the SRS follows ISO/IEC/IEEE 29148, quality follows ISO/IEC 25010, architecture uses C4, and privacy is handled by design.

6. Business analysis output

The work began with understanding the stakeholders and the current situation, then turned that understanding into requirements that can be measured.

Output	Document	Substance
Business requirements	PRODUCT-BRIEF	6 problems, 5 goals, scope and measures
Stakeholder view	STAKEHOLDER-MATRIX	9 groups with power, interest and a communication plan
Current and target state	AS-IS-TO-BE	6 pain points and 7 gaps, tied to requirements
Requirements	SRS	15 FRs and 8 NFRs, each with a verification method
User stories	USER-STORIES	10 stories with Given/When/Then acceptance criteria
Use cases	USE-CASES	6 cases with main, alternative and exception paths
Business rules	BUSINESS-RULES	10 rules in three groups

Output	Document	Substance
Traceability	TRACEABILITY-MATRIX	23 rows linking requirements to test cases

7. System analysis output

Requirements were turned into models and specifications a development team can act on.

Output	Document	Substance
Process maps	PROCESS-FLOWS	3 core processes as swimlanes
Architecture	C4-ARCHITECTURE	Context, container and component views
Data structure	DATA-MODEL	Entry table, vector index and row-level isolation policies
Sequence diagrams	SEQUENCE-DIAGRAMS	6 paths, including the failure path
Personal data flow	DFD-PRIVACY	Trust boundaries and the points where data leaves
Screen structure	UI-WIREFRAMES	6 screens with the real palette and fonts
API contract	API-CONTRACT	15 endpoints with example payloads
Decision records	ADR	11 architecture decisions

8. Quality assurance

Quality was made measurable, mapped to ISO/IEC 25010, then backed by risk assessment, a test plan and acceptance preparation.

Output	Document	Substance
Quality requirements	NFR-SPEC	8 areas written as measurable scenarios
Risk register	RISK-REGISTER	13 risks with mitigation and owners
Test plan	TEST-PLAN	4 levels with 24 test cases
Acceptance testing	UAT	7 scenarios with criteria and sign-off

9. Delivery

Output	Document	Substance
Delivery plan	ROADMAP	3 releases with MoSCoW priorities
Change log	CHANGE-REQUEST-LOG	9 real requests (CR-01 to CR-09) with impact analysis
Glossary	GLOSSARY	Codes and technical terms in one place

10. Results

The finished system works. It covers all 15 functional requirements, from writing and semantic retrieval through to summarising a mood as text and turning it into an image that keeps an important period. The document set traces 100 percent of requirements to test cases. Semantic retrieval runs on a vector index inside a single database, and personal data is isolated by database-level policies that cannot see across users.

Tangible outputs for a portfolio:

Output	Detail	Location
25 documents	Markdown is the source of truth, with a PDF for each (A4, Cordia New)	docs/ and output/pdf/
22-slide deck	Problem, architecture, data, privacy, quality and delivery plan, with speaker notes	output/presentation/
30-second clip	Tells the project story with real screens and diagrams from the documents	output/clip/
Working system	Frontend and API deployed separately on Vercel, with a Supabase database	rag-diary-app.vercel.app and rag-diary-api.vercel.app

11. Limitations

This work comes from a single person acting in simulated roles, and the imaginary team is small. The timing figures quoted are targets set from practice, not from field measurement, and testing with real users has not started yet. The system is genuinely deployed on Vercel and the main data paths have been tested.

Two limitations remain: the image generation quota on the AI provider is not available on the current plan, and user acceptance testing has not run. The client-side token, the missing rate limit and the absent CI test suite were closed in v1.1 and are recorded in RISK-REGISTER and ROADMAP.

12. Lessons and what I would do better

The clearest lesson is that security has to be designed in from the start, not collected afterwards. Keeping the token on the client and having no rate limit were things I only later understood should be done differently. The system records both as improvements and closes them in v1.1 through ADR-10 and the rate_limits table.

A second lesson is that quality has to be written so it can be measured. If quality is to carry weight, it must be defined as a number that can be checked, not as a vague word like "fast" or "secure". Once it is a measurable scenario, the test team knows what passing means.

13. Document links

Links marked (TH) point to the Thai edition until that document is translated.

Group	Documents
Package	README, DOCUMENT-PLAN (TH), CASE-STUDY (this document)
Discovery	PRODUCT-BRIEF (TH), STAKEHOLDER-MATRIX (TH), AS-IS-TO-BE (TH)
Requirements	SRS (TH), USER-STORIES (TH), USE-CASES (TH), BUSINESS-RULES (TH), TRACEABILITY-MATRIX (TH)
Modelling	PROCESS-FLOWS (TH), C4-ARCHITECTURE (TH), DATA-MODEL (TH), SEQUENCE-DIAGRAMS (TH), DFD-PRIVACY (TH)
Design	UI-WIREFRAMES (TH), API-CONTRACT (TH), ADR (TH)
Quality	NFR-SPEC (TH), RISK-REGISTER (TH), TEST-PLAN (TH), UAT (TH)
Delivery	ROADMAP (TH), CHANGE-REQUEST-LOG (TH), GLOSSARY (TH)

> Everything lives in the project's docs folder, with DOCUMENT-PLAN as the central register.